PocketToons analytics agent: write-up

What I built and why

An **LLM-neutral investigation framework** that a coding agent (Claude Code or Codex) runs against the SQLite database. There is no custom orchestration code: the agent's own reasoning loop is the runtime, and the project supplies what makes it reliable.

- **Shared knowledge:** a data catalog and a **metric registry** with approved, versioned definitions and canonical SQL, so "MAU" or "net revenue" means one thing regardless of model.
- **Layered data:** 20 Bronze views, 10 Silver and 8 Gold tables built in one transaction with 122 reconciliation checks. Gold answers standard questions fast; Silver and Bronze keep the detail for investigations and audits.
- **Guardrails as tools:** a read-only query tool (SQLite authorizer, row and time limits), an evidence tool that writes one log per query and never overwrites, and a fixed-template PDF generator that refuses reports over 10 pages.
- **A shared skill** (`SKILL.md`) for the method: clarify, check coverage, choose a layer, test bounded hypotheses, and separate observation from cause.

Why: definitions and guardrails live in files, not in a prompt, so they work across models, can be reviewed, and improve when a failure becomes a rule. *Trade-off:* no UI, and quality depends on the agent following the skill.

Tools: Claude Code (Claude Opus 5.5, medium effort) built the framework, ran the GM investigation and computed the answer key. Codex (GPT Terra 6.1, medium) ran 17 of the 18 evaluation runs, so the agent was tested in a different runtime from the one that built it. ChatGPT (Terra 5.6, later Terra 6.1, medium) acted as an independent reviewer of the GM report and the evaluation decisions.

Where it breaks

18 blind scenarios scored against an answer key computed from source tables only: **15 pass, 1 partial, 2 fail** in round 1; one new rule made it **16 / 1 / 1**, with no regressions (results).

- **Overconfidence** (GM report v1, the agent's raw output; v2 is the agent's revision after review feedback): a cause stated from an association, hypotheses rejected too firmly, mixed comparison periods. Fixed with skill rules; it didn't recur in 17 later runs.
- **Missing context the data doesn't announce:** April is a start-up month (the subscriber base starts at zero), and the agent used it as a normal baseline (Q07, Q11). A rule fixed Q07 but not Q11, where the question isn't phrased as a comparison. **This is the hardest question type.**
- **Noise:** every run repeated a harmless registry-hash warning, and simple questions got 5-page reports.
- **Strongest area:** ambiguity. It asked in all four scenarios that needed it, declined the unanswerable question, and caught both planted data traps.

Running it for a 20-person product team

- **Access:** read-only role on a warehouse replica; the query tool's limits stay on; PII columns excluded from the catalog (not implemented yet).
- **Latency (measured):** 2–5 minutes for simple questions, about 11 minutes question-to-report for the multi-step GM question; each query took under 2 seconds.
- **Cost:** not measured in this project. Needs token logging per run before setting a budget per question.
- **Quality over time:** keep the 18 scenarios as a regression suite, rerun on every change to the skill, registry or data model, and track pass rate, clarification rate and query errors. Sample real questions each week for human review: failure → general rule → rerun, the loop demonstrated here.
- **Errors:** evidence logs make every number traceable; a failed or truncated query is reported, never treated as zero.

With more time

1. Rephrase the start-up rule so it also covers trend descriptions (Q11), and verify definitions by comparing SQL rather than file hashes.
2. Run the same questions in both runtimes to compare Claude and Codex directly, and add held-out questions.
3. Measure tokens and cost per run; add a short-answer mode for simple questions.
4. Rerun the GM question on the improved agent; a machine-readable catalog; a simple UI.